3. The "Rendezvous Point" – 25 pts (Wet)

Scenario: Two emergency response teams are located at different nodes in a city graph: Team Alpha at node $n1$ and Team Bravo at node $n2$. They need to meet in a point R, a Rendezvous Point to consolidate equipment before a mission.

Objective: Find the node R such that the time it takes for both teams is minimized.

1. (7 pts) Provide the pseudocode of an algorithm of a function `findRendezvousPoint(n1, n2)` that solves the problem (i.e., return the node R).

   Solution

```java
findRendezvousPoint(n1, n2)
 assert n1 and n2 not null
    Map distTo1 = new_map()
    Map distTo2 = new_map()
    call dijkstraShortestPath(Graph G, n1)
    call dijkstraShortestPath(Graph G, n2)
    //dijkstraShortestPath will fill the disTo map with shortest path
    int bestDist = infty
    int candidate = null
    for each v in graph do
        if(max(distTo1[v],distTo2[v]) < bestDist)
            then bestDist = max(distTo1[v],distTo2[v])
                 candidate = v
        end if
    end for
    return candidate
```

2. (8 pts) Show the algorithm is correct and provide its complexity. If your algorithm relies partially of algorithms given in the lecture, show can assume that those algorithms works correctly.

   Solution

   Here is the pseudocode, we analyze its complexity first.

   From lecture we know runtime complexity of `dijkstraShortestPath` is $O(|V|log|V| + |E|log|V|)$ where $|V|$ is the number of vertices and $|E|$ is the number of edges

   Then

```java
findRendezvousPoint(n1, n2)
 assert n1 and n2 not null
    //O(1) since we can build hash map
    Map distTo1 = new_map()
    Map distTo2 = new_map()
    // O(|V|log|V| + |E|log|V|)
    call dijkstraShortestPath(Graph G, n1)
    // O(|V|log|V| + |E|log|V|)
    call dijkstraShortestPath(Graph G, n2)
    //dijkstraShortestPath will fill the disTo map with shortest path
    //O(1)
    int bestDist = infty
    int candidate = null
    //O(|V|) = O(|V|) * O(1)
    for each v in graph do
        //O(1)
        if(max(distTo1[v],distTo2[v]) < bestDist)
            then bestDist = max(distTo1[v],distTo2[v])
                 candidate = v
        end if
    end for
    return candidate
```

Thus the complexity of `findRendezvousPoint` is $O(|V|log|V| + |E|log|V|)$

Then we prove this algorithm is correct and suppose `dijkstraShortestPath` works correctly

1. Since `dijkstraShortestPath` works correctly and it will fill the `disTo` map with shortest path, then after execute these four steps

   ```java
   Map distTo1 = new_map()
   Map distTo2 = new_map()
   call dijkstraShortestPath(Graph G, n1)
   call dijkstraShortestPath(Graph G, n2)
   ```

   We are guaranteed that `distTo1` is the map s.t. shortest path from n1 to that node is stored in it correctly. The same for `distTo2`

2. Then we execute those steps

```java
int bestDist = infty
int candidate = null
for each v in graph do
    if(max(distTo1[v],distTo2[v]) < bestDist)
        then bestDist = max(distTo1[v],distTo2[v])
             candidate = v
    end if
end for
```

In the beginning, we assume `bestDist` is infinity and `candidate` is null

We want to find the node R such that the time it takes for both teams is minimized, this is equivalent to finding the node $R = v,\ \text{where } v \text{ satisfy } \max(\text{distTo1}[v], \text{distTo2}[v]) \leq \max(\text{distTo1}[u], \text{distTo2}[u]), \forall u \in \text{graph}.$

Because two teams go to R parallelly, then the time of meeting at R is the maximum of two teams' cost time. And suppose the speed of two teams are same, then we only need to find the minimum of maximum of two teams' distance to R

To find minimum of maximum of two teams' distance to R, we need two teams' distance to R are both shortest. This is guaranteed by the algorithm of `dijkstraShortestPath`.

Thus we only need to compare and find the minimum one.

At first loop, the `max(distTo1[v],distTo2[v])` must be less than infinity since the graph is connected and not empty. Then `bestDist` and `candidate` are updated successfully.

Then once we find a distance that the maximum distance of n1/n2 to v is shorter than `bestDist`, we can update `bestDist` and `candidate`.

Thus in the end, we will find such `candidate` is R and return it

3. (10 pts) Implement the algorithm in Java. Provide meaningful tests.

Finished

Hint: Think how Dijkstra could help.